Practical-1

```
import numpy as np
import pandas as pd
data = np.array([['','Col1','Col2'], ['Row1',1,2],
['Row2',3,4]])
print(pd.DataFrame(data=data[1:,1:],
index = data [1:,0], columns=data[0,1:]))
# Take a 2D array as input to your DataFrame
my_2darray = np.array([[1, 2, 3], [4, 5, 6]])
print(pd.DataFrame(my_2darray))
# Take a dictionary as input to your DataFrame
my_dict = {1: ['1', '3'], 2: ['1', '2'], 3: ['2', '4']}
print(pd.DataFrame(my_dict))
# Take a DataFrame as input to your DataFrame
my_df = pd.DataFrame(data=[4,5,6,7], index=range(0,4), columns=['A'])
print(pd.DataFrame(my_df))
# Take a Series as input to your DataFrame
my_series = pd.Series({"United Kingdom":"London", "India":"New Delhi", "United
States":"Washington", "Belgium":"Brussels"})
print(pd.DataFrame(my_series))
df = pd.DataFrame(np.array([[1, 2, 3], [4, 5, 6]]))
# Use the `shape` property print(df.shape)
# Or use the `len()` function with the `index` property
print(len(df.index))
```

Practical-2: Implement Python Program for NumPy Arrays

```
import numpy as np
arr = np.array([[1, 2, 3],
[4, 2, 5]])
print("Array is of type: ", type(arr))
print("No. of dimensions: ", arr.ndim)
print("Shape of array: ", arr.shape)
print("Size of array: ", arr.size)
print("Array stores elements of type: ", arr.dtype)
```

Practical-2.1: More about NumPy Arrays and data frames

```
import numpy as np
import pandas as pd
data = np.array([['','Col1','Col2'], ['Row1',1,2],
['Row2',3,4]])
print(pd.DataFrame(data=data[1:,1:],
index = data[1:,0], columns=data[0,1:]))
my_2darray = np.array([[1, 2, 3], [4, 5, 6]])
print(pd.DataFrame(my_2darray))
my_dict = {1: ['1', '3'], 2: ['1', '2'], 3: ['2', '4']}
print(pd.DataFrame(my_dict))
my_df = pd.DataFrame(data=[4,5,6,7], index=range(0,4), columns=['A'])
print(pd.DataFrame(my_df))
my_series = pd.Series({"United Kingdom":"London", "India":"New Delhi",
"United States":"Washington", "Belgium":"Brussels"})
print(pd.DataFrame(my_series))
df = pd.DataFrame(np.array([[1, 2, 3], [4, 5, 6]]))
print(len(df.index))
```

Lab Manual 3

```
# Creating DataFrames (Multiple Methods)
import pandas as pd
data = {
'ID': [1, 2, 3, 4],
'Name': ['Alice', 'Bob', 'Charlie', 'David'],
'Score': [78, 85, 90, 88]
}
df = pd.DataFrame(data)
print(df)
data2 = [[101, 'Math', 85], [102, 'Science', 90]]
df2 = pd.DataFrame(data2, columns=['Roll', 'Subject', 'Marks'])
print(df2)
# Inspecting and Understanding Data
print(df.head())
print(df.tail())
print(df.shape)
print(df.info())
print(df.describe())
# Row and Column Operations
df['Bonus'] = 5
df['Final_Score'] = df['Score'] + df['Bonus']
print(df)
# Drop row and column
df.drop(3, axis=0, inplace=True)
df.drop('Bonus', axis=1, inplace=True)
print(df)
# Filtering and Conditional Selection
high_scores = df[df['Final_Score'] > 85]
print(high_scores)
# Multiple conditions
filtered = df[(df['Final_Score'] > 80) & (df['ID'] > 1)]
```

```
print(filtered)
# Sorting and Ranking
print(df.sort_values(by='Final_Score', ascending=False))
df['Rank'] = df['Final_Score'].rank(ascending=False)
print(df)
# Working with Missing Data
import numpy as np
df.loc[1, 'Final_Score'] = np.nan
print(df.isnull())
print(df.fillna(df['Final_Score'].mean()))
print(df.dropna())
# Renaming Columns and Index
df.rename(columns={'Final_Score': 'Total'}, inplace=True)
df.index = ['A', 'B', 'C']
print(df)
# String Operations
df['Name_Upper'] = df['Name'].str.upper()
df['Name_Length'] = df['Name'].str.len()
print(df)
#GroupBy Operations
data3 = {
'Dept': ['IT', 'IT', 'HR', 'HR'],
'Employee': ['A', 'B', 'C', 'D'],
'Salary': [50000, 60000, 45000, 48000] }
emp_df = pd.DataFrame(data3)
print(emp_df.groupby('Dept')['Salary'].mean())
# Applying Functions
df['Grade'] = df['Total'].apply(lambda x: 'A' if x >= 90 else 'B')
print(df)
# Exporting Data
df.to_csv('output.csv', index=False)
print("Data exported successfully")
```

Practical-4

write a program Reading data from text files, Excel and the web and exploring various commands for doing descriptive analytics on the Iris data set

```
import pandas as pd

import numpy as np

#Upload CSV File (Colab only)

from google.colab import files

files.upload()

# Read CSV File

 #Upload Excel File

files.upload()

# Read Excel File

df_excel = pd.read_excel("iris.xlsx")

df_excel.head()

# Load Data Directly from URL

url = "https://raw.githubusercontent.com/mwaskom/seaborn

data/master/iris.csv"

df_web = pd.read_csv(url)

df_web.head()

df_web.shape

df_web.describe()

df_web.mean(numeric_only=True)

df_web.median(numeric_only=True)

df_web.mode()

df_web.std(numeric_only=True)

df_web.var(numeric_only=True)

df_web.skew(numeric_only=True)

df_web.kurtosis(numeric_only=True)

df_web['species'].value_counts()

df_web.groupby('species').mean()

df_web.corr(numeric_only=True)

df_web.isnull().sum()

df_web[['sepal_length', 'petal_length']].head()
```

#P5

Pima dataset import

A. Univariate analysis: Frequency, Mean, Median, Mode, Variance, Standard Deviation, Skewness and Kurtosis.

```
import  pandas as pd

import matplotlib.pyplot as plt

import seaborn as sns

from sklearn.model_selection import train_test_split

from sklearn.linear_model import LinearRegression, LogisticRegression

from sklearn.metrics import accuracy_score, mean_squared_error, r2_score, confusion_matrix


url = "https://raw.githubusercontent.com/jbrownlee/Datasets/master/pima-indians-diabetes.data.csv"

columns = ['Pregnancies', 'Glucose', 'BloodPressure', 'SkinThickness', 'Insulin', 'BMI', 'DiabetesPedigreeFuncton', 'Age', 'Outcome']

df_pima = pd.read_csv(url)

df_pima = pd.read_csv(url, names=columns)

df_pima.head()

from sklearn.datasets import load_diabetes

diabetes = load_diabetes()

df_uci = pd.DataFrame(diabetes.data,columns=diabetes.feature_names)

df_uci['target'] = diabetes.target

print(df_uci.head())


df_pima.describe()

df_pima.mean(numeric_only=True)

df_pima.median(numeric_only=True)


df_pima.mode()

df_pima.std(numeric_only=True)


df_pima.var(numeric_only=True)

for col in df_pima.columns[:-1]:
```

```
print(df_pima[col].value_counts())

df_pima.skew(numeric_only=True)

df_pima.kurtosis(numeric_only=True)

# Example for Pima
for col in df_pima.columns[:-1]: # exclude Outcome
  sns.histplot(df_pima[col], kde=True)
  plt.title(f'Pima: {col}')
  plt.show()
```

 # Bivariate Analysis – Pima - Correlation Heatmap

```
# Correlation heatmap - numeric Vs numeric
plt.figure(figsize=(10,6))
sns.heatmap(df_pima.corr(), annot=True,
cmap='coolwarm')
plt.title("Correlation between Variables")
plt.show()
# Loop through all columns except 'Outcome'
for col in df_pima.columns[:-1]:
  plt.figure(figsize=(6, 4))
  sns.boxplot(x='Outcome', y=col, data=df)
  plt.title(f'{col} vs Outcome (Pima)')
  plt.show()
```

```
# Bivariate Linear Regression
X = df_pima[['Glucose']]
y = df_pima['Outcome']

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)
lin_model = LinearRegression()
lin_model.fit(X_train, y_train)

y_pred = lin_model.predict(X_test)

print("Coefficient:", lin_model.coef_[0])
print("Intercept:", lin_model.intercept_)
print("R² score:", r2_score(y_test, y_pred))

plt.scatter(X, y, alpha=0.5)
plt.plot(X, lin_model.predict(X), color='red')
plt.xlabel("Glucose")
plt.ylabel("Outcome")
plt.title("Linear Regression: Glucose vs Outcome")
plt.show()

# Bivariate Logistic Regression
log_model = LogisticRegression()
log_model.fit(X_train, y_train)

y_pred = log_model.predict(X_test)

print("Coefficient:", log_model.coef_[0][0])
print("Accuracy:", accuracy_score(y_test, y_pred))
```

```
X_range = np.linspace(X.min(), X.max(), 100).reshape(-1, 1)
y_prob = log_model.predict_proba(X_range)[:, 1]

plt.scatter(X, y, alpha=0.4)
plt.plot(X_range, y_prob, color='red')
plt.xlabel("Glucose")
plt.ylabel("Probability of Diabetes")
plt.title("Logistic Regression: Glucose vs Outcome")
plt.show()
```

 # MULTIPLE REGRESSION ANALYSIS

```
features = ['Glucose', 'BMI', 'Age', 'Pregnancies']
X = df_pima[features]
y = df_pima['Outcome']

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)
multi_lin_model = LinearRegression()
multi_lin_model.fit(X_train, y_train)

y_pred = multi_lin_model.predict(X_test)

coefficients = pd.DataFrame(multi_lin_model.coef_, index=features,
columns=['Coefficient'])
print(coefficients)
print("R² score:", r2_score(y_test, y_pred))

coefficients.plot(kind='bar')
plt.title("Multiple Linear Regression Coefficients")
plt.ylabel("Coefficient Value Or features importance")
plt.show()
```

```
multi_log_model = LogisticRegression(max_iter=1000)
multi_log_model.fit(X_train, y_train)

y_pred = multi_log_model.predict(X_test)

coefficients = pd.DataFrame(multi_log_model.coef_[0], index=features,
columns=['Coefficient'])
print(coefficients)
print("Accuracy:", accuracy_score(y_test, y_pred))

coefficients.plot(kind='bar', color='orange')
plt.title("Multiple Logistic Regression Coefficients")
plt.ylabel("Coefficient Value")
plt.show()
```

UCI dataset

 # A. Univariate analysis: Frequency, Mean, Median, Mode, Variance, Standard Deviation, Skewness and Kurtosis.

#same as PIMA REPLACE df_pima with df_uci

#Distribution visualisation

```
for col in df_uci.columns[:-1]: # exclude Outcome
  sns.histplot(df_uci[col], kde=True)
  plt.title(f'UCI: {col}')
  plt.show()
```

Bivariate Analysis – UCI - Correlation Heatmap

 # Correlation heatmap - numeric Vs numeric

```
plt.figure(figsize=(10,6))
sns.heatmap(df_uci.corr(), annot=True,cmap='coolwarm')
plt.title("Correlation between Variables")
plt.show()
```

#Box Plots for UCI Diabetes Dataset

```
#Create target bins (e.g., Low, Medium, High)
df_uci['target_bin'] = pd.qcut(df_uci['target'], q=3, labels=['Low', 'Medium', 'High'])
for col in df_uci.columns[:-2]:  # exclude 'target' and 'target_bin'
  plt.figure(figsize=(6, 4))
  sns.boxplot(x='target_bin', y=col, data=df_uci)
  plt.title(f'{col} vs Target Bin (UCI)')
  plt.show()
```

Bivariate Linear Regression

```
X_bmi = df_uci[['bmi']]
y = df_uci['target']

X_train, X_test, y_train, y_test = train_test_split(
    X_bmi, y, test_size=0.2, random_state=42
)
model_lr = LinearRegression()
model_lr.fit(X_train, y_train)

y_pred = model_lr.predict(X_test)

plt.figure(figsize=(6,4))
plt.scatter(X_test, y_test, color="blue", label="Actual values")
plt.plot(X_test, y_pred, color="red", label="Regression line")
plt.xlabel("BMI")
plt.ylabel("Target")
plt.title("Linear Regression: BMI vs Target")
plt.legend()
plt.show()
```

```
mse = mean_squared_error(y_test, y_pred)
r2 = r2_score(y_test, y_pred)

print("MSE:", mse)
print("R² Score:", r2)

# MULTIPLE REGRESSION ANALYSIS
model_multi = LinearRegression()
model_multi.fit(X_train, y_train)

y_pred_multi = model_multi.predict(X_test)

mse_multi = mean_squared_error(y_test, y_pred_multi)
r2_multi = r2_score(y_test, y_pred_multi)

print("Multiple Regression MSE:", mse_multi)
print("Multiple Regression R²:", r2_multi)

plt.figure(figsize=(6,4))
plt.scatter(y_test, y_pred_multi)
plt.xlabel("Actual Target")
plt.ylabel("Predicted Target")
plt.title("Actual vs Predicted Values (Multiple Regression)")
plt.plot([y_test.min(), y_test.max()], [y_test.min(), y_test.max()], color="red")
plt.show()
```

#P6

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from scipy.stats import norm
from mpl_toolkits.mplot3d import Axes3D
from sklearn.datasets import load_diabetes

diabetes=load_diabetes()
df=pd.DataFrame(diabetes.data,columns=diabetes.feature_names)
df['target'] = diabetes.target
print(df.head())

df.shape
df.info()
df.describe()

# A Normal Curve Fit For BMI Feature.
data = df['bmi']
mu, std = norm.fit(data)
plt.hist(data,bins=20,density=True)
xmin,xmax = plt.xlim()
x=np.linspace(xmin,xmax,100)
p=norm.pdf(x,mu,std)
plt.plot(x,p)
plt.title("Normal Curve Fit For BMI Feature")
plt.show()
```

```
#Density and Contour Plots
sns.kdeplot(df['bp'],fill=True)
plt.title("Density plot of Blood Pressure Feature")
plt.show()
sns.kdeplot(x=df['bmi'],
        y=df['bp'],
        fill=True)
plt.title("Contour Plot: BMI VS BP")
plt.show()

#Correlation and Scatter Plots
plt.scatter(df['bmi'],df['target'])
plt.title("Scatter Plot: BMI vs Disease Progression Target")
plt.xlabel("BMI")
plt.ylabel("Target")
plt.show()
plt.figure(figsize=(10,6))
sns.heatmap(df.corr(),annot=True)
plt.title("Corrleation Heatmap - Diabetes Dataset")
plt.show()

#Histogram PLooting
plt.hist(df['age'],bins=15)
plt.title("Histogram of Age Features")
plt.xlabel("Age")
plt.ylabel("Frequency")
plt.show()

#Multi-Histogram
df[['age','bmi','bp']].hist(bins=15,figsize=(10,6))
plt.show()
```

```
#Three Dimensional Plooting

fig=plt.figure()
ax = fig.add_subplot(111,projection='3d')
ax.scatter(
    df['bmi'],
    df['bp'],
    df['target']
)
ax.set_xlabel("BMI")
ax.set_ylabel("Blood Pressure")
ax.set_zlabel("target")
plt.title("3D Scatter Plot: BMI,BP and Target")
plt.show()
```

#P7

dataset: https://www.kaggle.com/datasets/aliiihussain/car-price-prediction?resource=download

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import LabelEncoder, StandardScaler
from sklearn.linear_model import LinearRegression
from sklearn.metrics import r2_score, mean_absolute_error

# # 1. Load Dataset & datset summary
```

```
df = pd.read_csv("/content/car_price_prediction_.csv")
df.head()
df.info()
df.describe()

# ### How to find unique values of Brand?
# How to print only "Toyota" brand car details from the dataset

# df['Brand'].unique()
# df['Condition'].unique()
df[df['Brand'] == 'Audi']



# # 2. Basic Cleaning steps
df.isnull().sum()
# df.dropna(inplace=True)

# # 3. Convert Categorical Columns
le = LabelEncoder()
for col in ["Brand", "Model", "Fuel Type", "Transmission"]:
    df[col] = le.fit_transform(df[col])
df.head()

# # 4. Exploratory Data Analysis
#Univariate EDA

plt.hist(df["Price"], bins=20)
plt.title("Distribution of Car Prices")
plt.xlabel("Price")
plt.ylabel("Frequency")
plt.show()
```

```
sns.boxplot(y=df["Price"])
plt.title("Box Plot of Price")
plt.show() #outliers

sns.boxplot(y=df["Mileage"])
plt.title("Box Plot of Mileage")
plt.show()

# Relationship Analysis (Bivariate EDA)

plt.scatter(df["Mileage"], df["Price"])
plt.title("Scatter: Mileage vs Price")
plt.xlabel("Mileage")
plt.ylabel("Price")
plt.show()

# # 5. Categorical Analysis
sns.boxplot(x="Fuel Type", y="Price", data=df)
plt.show()

# # 6. Correlation Analysis
sns.heatmap(df.corr(), annot=True)
plt.show()
df['Condition'].unique()
le1 = LabelEncoder()
df['Condition'] = le1.fit_transform(df['Condition'])
df.columns
df.drop(columns=['Car ID'], inplace=True)
fig, ax = plt.subplots(figsize=(12, 10))
sns.heatmap(df.corr(), annot=True, ax=ax)
```

```
plt.show()
# plt.figure(figsize=(8,6))
# sns.heatmap(df.corr(), annot=True)
# plt.show()

# # 7. Build Regression Model

X = df.drop("Price", axis=1)
y = df["Price"]
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)
model = LinearRegression()
model.fit(X_train, y_train)
y_pred = model.predict(X_test)

# # 8. Evaluate Model

print("R2 Score:", r2_score(y_test, y_pred))
print("MAE:", mean_absolute_error(y_test, y_pred))
```

#P8

```
import numpy as np
import pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt

# # **Exploring Data**
```

```
# dataset from:https://www.kaggle.com/datasets/rikdifos/credit-card-approval-prediction/data


credit_record_df = pd.read_csv('/content/credit_record.csv')

credit_record_df.head(20)

application_record_df = pd.read_csv('/content/application_record.csv')

application_record_df.head()


# #### Size of Data

print(f'Credit data have {credit_record_df.shape[0]} rows , {credit_record_df.shape[1]} columns.')

print(f'Application data have {application_record_df.shape[0]} rows , {application_record_df.shape[1]} columns.')



# #### Cecking NULL values

print('Credit data null values:')

print(credit_record_df.isnull().sum())

print('Application data null values:')

print(application_record_df.isnull().sum())


# #### Checking Data Types

credit_record_df.info()

credit_record_df['STATUS'].unique()

application_record_df.info()


# # **Relationships between Columns**

sns.countplot(x='STATUS', data=credit_record_df)

plt.show()

plt.figure(figsize=(15, 6))

sns.countplot(x='MONTHS_BALANCE', data=credit_record_df)

plt.show()
```

```
credit_record_df.MONTHS_BALANCE.nunique()

credit_record_df.MONTHS_BALANCE.min()# showing when was the last time the client got month money into his account

credit_record_df.info()


#credit_record_df['MONTHS_BALANCE'].groupby(credit_record_df['STATUS']).median()

credit_record_df['MONTHS_BALANCE'].groupby(credit_record_df['STATUS']).agg(pd.Series.mode)


plt.figure(figsize=(10, 6))

sns.histplot(credit_record_df['MONTHS_BALANCE'], bins=30, kde=True)

plt.title('Distribution of MONTHS_BALANCE')

plt.xlabel('Months Balance')

plt.ylabel('Frequency')

plt.show()


# sns.countplot(data=credit_record_df,x='STATUS', hue=credit_record_df['MONTHS_BALANCE'])


figsize = (10, 6)

plt.figure(figsize=figsize)

sns.countplot(x='NAME_INCOME_TYPE', data=application_record_df)

plt.show()


# we are adding new column to categize bank customer into Good and Bad


credit_record_df['Good or Bad'] = ['Good' if x in set(['0','X','C']) else 'Bad' for x in credit_record_df['STATUS']]

credit_record_df


credit_goods_bads = credit_record_df.groupby(['ID','Good or Bad']).size().to_frame('size')

credit_goods_bads.reset_index(inplace=True)
```

```
credit_goods_bads

idx = credit_goods_bads.groupby('ID')['size'].idxmax()
max_goods_bads = credit_goods_bads.loc[idx]
max_goods_bads

# print(hello)
max_goods_bads['Label'] = [1 if x == 'Good' else 0 for x in max_goods_bads['Good or Bad']]
max_goods_bads

max_goods_bads = max_goods_bads.drop(['Good or Bad', 'size'], axis=1)
max_goods_bads = max_goods_bads.reset_index(drop=True)
max_goods_bads
# #### **Time to join data**
# filling OCCUPATION_TYPE in application data
application_record_df.fillna('Unknown', inplace=True)

len(set(application_record_df['ID']).intersection(set(max_goods_bads['ID'])))
data = pd.merge(application_record_df, max_goods_bads, how='inner', on='ID')
data

# # **Preprocessing**
# ## **Feature Selection**

X = data.drop('Label', axis=1)
y = data['Label']
X

# ### **Mutual Information**
X.info()
X_info = X.copy()
```

```
# Label encoding for categoricals
for colname in X_info.select_dtypes("object"):
    X_info[colname], _ = X_info[colname].factorize()

# Converting floats to ints
for colname in X_info.select_dtypes("float"):
    X_info[colname] = X_info[colname].astype(int)

# Converting bools to ints
#for colname in X_info.select_dtypes("bool"):
#    X_info[colname] = X_info[colname].astype(int)



# All discrete features should now have integer dtypes
discrete_features = X_info.dtypes != int
from sklearn.feature_selection import mutual_info_regression

def make_mi_scores(X, y, discrete_features):
    mi_scores = mutual_info_regression(X, y, discrete_features=discrete_features)
    mi_scores = pd.Series(mi_scores, name="MI Scores", index=X.columns)
    mi_scores = mi_scores.sort_values(ascending=False)
    return mi_scores

mi_scores = make_mi_scores(X_info, y, discrete_features)
mi_scores

def plot_mi_scores(scores):
    scores = scores.sort_values(ascending=True)
    width = np.arange(len(scores))
    ticks = list(scores.index)
    plt.barh(width, scores)
```

```
    plt.yticks(width, ticks)
    plt.title("Mutual Information Scores")

plt.figure(dpi=100, figsize=(10, 15))
plot_mi_scores(mi_scores)

sns.relplot(x="AMT_INCOME_TOTAL", y="Label", data=data);

sns.lmplot(x="DAYS_EMPLOYED", y="Label", hue="CNT_CHILDREN", data=data);

#sns.lmplot(x="CODE_GENDER", y=y, hue="FLAG_OWN_REALTY", data=X_info);
plt.figure(dpi=50, figsize=(35, 10))
sns.scatterplot(x=data['OCCUPATION_TYPE'], y=data['Label'], hue=data['CODE_GENDER'])

# ## **Covert Categorical Data To Numerical**

X = pd.get_dummies(X, drop_first=True)
X

# ## **Scalling the Data**
from sklearn.model_selection import train_test_split
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)
from sklearn.preprocessing import StandardScaler
scaler = StandardScaler()
X_train_scaled= scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)
print("Normalized input data(X):\n", X_train_scaled)

# # **Modeling**
import pandas as pd
from sklearn.model_selection import train_test_split
```

```
from sklearn.metrics import accuracy_score, confusion_matrix, ConfusionMatrixDisplay
from sklearn.ensemble import RandomForestClassifier
from xgboost import XGBClassifier
import matplotlib.pyplot as plt

rf_model = RandomForestClassifier(n_estimators=100, random_state=42)
rf_model.fit(X_train, y_train)
rf_pred = rf_model.predict(X_test)
rf_cm = confusion_matrix(y_test, rf_pred)
print("Random Forest Confusion Matrix")
print(rf_cm)


ConfusionMatrixDisplay(confusion_matrix=rf_cm).plot()
plt.title("Random Forest Confusion Matrix")
plt.show()

xgb_model = XGBClassifier(use_label_encoder=False, eval_metric='logloss')
xgb_model.fit(X_train, y_train)
xgb_pred = xgb_model.predict(X_test)
xgb_cm = confusion_matrix(y_test, xgb_pred)
print("XGBoost Confusion Matrix")
print(xgb_cm)

ConfusionMatrixDisplay(confusion_matrix=xgb_cm).plot()
plt.title("XGBoost Confusion Matrix")
plt.show()

# ### **Using Stratified K-Fold with Decision Tree Classifier**

# from sklearn.tree import DecisionTreeClassifier
```

```
# from sklearn.model_selection import StratifiedKFold, cross_val_score, cross_val_predict
# from sklearn.metrics import confusion_matrix, ConfusionMatrixDisplay
# import matplotlib.pyplot as plt

# clf = DecisionTreeClassifier(random_state=42)

# sk_folds = StratifiedKFold(n_splits=10)

# # Cross-validation accuracy scores
# scores = cross_val_score(clf, X_train_scaled, y_train, cv=sk_folds)

# print("Cross Validation Scores:", scores)
# print("Average CV Score:", scores.mean())
# print("Number of CV Scores used in Average:", len(scores))
# # Get predictions using cross validation
# y_pred = cross_val_predict(clf, X_train_scaled, y_train, cv=sk_folds)
# # Confusion Matrix
# cm = confusion_matrix(y_train, y_pred)
# print("Confusion Matrix:")
# print(cm)

# # Plot confusion matrix
# ConfusionMatrixDisplay(confusion_matrix=cm).plot()
# plt.title("Decision Tree Confusion Matrix (10-fold CV)")
# plt.show()
```